Flashsort

Flashsort is a distribution sorting algorithm showing linear computational complexity $O(n)$ for uniformly distributed data sets and relatively little additional memory requirement. The original work was published in 1998 by Karl-Dietrich Neubert.[1]

Concept

Flashsort is an efficient in-place implementation of histogram sort, itself a type of bucket sort. It assigns each of the n input elements to one of m *buckets*, efficiently rearranges the input to place the buckets in the correct order, then sorts each bucket. The original algorithm sorts an input array A as follows:

1. Using a first pass over the input or *a priori* knowledge, find the minimum and maximum sort keys.
2. Linearly divide the range $[A_{\min}, A_{\max}]$ into m buckets.
3. Make one pass over the input, counting the number of elements A_i which fall into each bucket. (Neubert calls the buckets "classes" and the assignment of elements to their buckets "classification".)
4. Convert the counts of elements in each bucket to a prefix sum, where L_b is the number of elements A_i in bucket b or less. ($L_0 = 0$ and $L_m = n$.)
5. Rearrange the input so all elements of each bucket b are stored in positions A_i where $L_{b-1} < i \leq L_b$.
6. Sort each bucket using insertion sort.

Steps 1–3 and 6 are common to any bucket sort, and can be improved using techniques generic to bucket sorts. In particular, the goal is for the buckets to be of approximately equal size (n/m elements each),[1] with the ideal being division into m quantiles. While the basic algorithm is a linear interpolation sort, if the input distribution is known to be non-uniform, a non-linear division will more closely approximate this ideal. Likewise, the final sort can use any of a number of techniques, including a recursive flash sort.

What distinguishes flash sort is step 5: an efficient $O(n)$ in-place algorithm for collecting the elements of each bucket together in the correct relative order using only m words of additional memory.

Memory efficient implementation

The Flashsort rearrangement phase operates in cycles. Elements start out "unclassified", then are

moved to the correct bucket and considered "classified". The basic procedure is to choose an unclassified element, find its correct bucket, exchange it with an unclassified element there (which must exist, because we counted the size of each bucket ahead of time), mark it as classified, and then repeat with the just-exchanged unclassified element. Eventually, the element is exchanged with itself and the cycle ends.

The details are easy to understand using two (word-sized) variables per bucket. The clever part is the elimination of one of those variables, allowing twice as many buckets to be used and therefore half as much time spent on the final $O(n^2)$ sorting.

To understand it with two variables per bucket, assume there are two arrays of m additional words: K_b is the (fixed) upper limit of bucket b (and $K_0 = 0$), while L_b is a (movable) index into bucket b, so $K_{b-1} \le L_b \le K_b$.

We maintain the loop invariant that each bucket is divided by L_b into an unclassified prefix (A_i for $K_{b-1} < i \le L_b$ have yet to be moved to their target buckets) and a classified suffix (A_i for $L_b < i \le K_b$ are all in the correct bucket and will not be moved again). Initially $L_b = K_b$ and all elements are unclassified. As sorting proceeds, the L_b are decremented until $L_b = K_{b-1}$ for all b and all elements are classified into the correct bucket.

Each round begins by finding the first incompletely classified bucket c (which has $K_{c-1} < L_c$) and taking the first unclassified element in that bucket A_i where $i = K_{c-1} + 1$. (Neubert calls this the "cycle leader".) Copy A_i to a temporary variable t and repeat:

- Compute the bucket b to which t belongs.
- Let $j = L_b$ be the location where t will be stored.
- Exchange t with A_j, i.e. store t in A_j while fetching the previous value A_j thereby displaced.
- Decrement L_b to reflect the fact that A_j is now correctly classified.
- If $j \ne i$, restart this loop with the new t.
- If $j = i$, this round is over and find a new first unclassified element A_i.
- When there are no more unclassified elements, the distribution into buckets is complete.

When implemented with two variables per bucket in this way, the choice of each round's starting point i is in fact arbitrary; *any* unclassified element may be used as a cycle leader. The only requirement is that the cycle leaders can be found efficiently.

Although the preceding description uses K to find the cycle leaders, it is in fact possible to do without it, allowing the entire m-word array to be eliminated. (After the distribution is complete, the bucket boundaries can be found in L.)

Suppose that we have classified all elements up to $i-1$, and are considering A_i as a potential new cycle leader. It is easy to compute its target bucket b. By the loop invariant, it is classified if $L_b < i \le K_b$, and unclassified if i is outside that range. The first inequality is easy to test, but the second appears to require the value K_b.

It turns out that the induction hypothesis that all elements up to $i-1$ are classified implies that $i \le K_b$, so it is not necessary to test the second inequality.

Consider the bucket c which position i falls into. That is, $K_{c-1} < i \le K_c$. By the induction hypothesis, all elements below i, which includes all buckets up to $K_{c-1} < i$, are completely classified. I.e. no elements which belong in those buckets remain in the rest of the array. Therefore, it is not possible that $b < c$.

The only remaining case is $b \ge c$, which implies $K_b \ge K_c \ge i$, Q.E.D.

Incorporating this, the flashsort distribution algorithm begins with L as described above and $i = 1$. Then proceed:[1][2]

- If $i > n$, the distribution is complete.
- Given A_i, compute the bucket b to which it belongs.
- If $i \le L_b$, then A_i is unclassified. Copy it a temporary variable t and:
  - Let $j = L_b$ be the location where t will be stored.
  - Exchange t with A_j, i.e. store t in A_j while fetching the previous value A_j thereby displaced.
  - Decrement L_b to reflect the fact that A_j is now correctly classified.
  - If $j \ne i$, compute the bucket b to which t belongs and restart this (inner) loop with the new t.
- A_i is now correctly classified. Increment i and restart the (outer) loop.

While saving memory, Flashsort has the disadvantage that it recomputes the bucket for many already-classified elements. This is already done twice per element (once during the bucket-counting phase and a second time when moving each element), but searching for the first unclassified element requires a third computation for most elements. This could be expensive if buckets are assigned using a more complex formula than simple linear interpolation. A variant reduces the number of computations from almost $3n$ to at most $2n + m - 1$ by taking the *last* unclassified element in an unfinished bucket as cycle leader:

- Maintain a variable c identifying the first incompletely-classified bucket. Let $c = 1$ to begin with, and when $c > m$, the distribution is complete.
- Let $i = L_c$. If $i = L_{c-1}$, increment c and restart this loop. ($L_0 = 0$.)
- Compute the bucket b to which A_i belongs.
- If $b < c$, then $L_c = K_{c-1}$ and we are done with bucket c. Increment c and restart this loop.
- If $b = c$, the classification is trivial. Decrement L_c and restart this loop.
- If $b > c$, then A_i is unclassified. Perform the same classification loop as the previous case, then restart this loop.

Most elements have their buckets computed only twice, except for the final element in each bucket, which is used to detect the completion of the following bucket. A small further reduction can be achieved by maintaining a count of unclassified elements and stopping when it reaches

zero.

Performance

The only extra memory requirements are the auxiliary vector L for storing bucket bounds and the constant number of other variables used. Further, each element is moved (via a temporary buffer, so two move operations) only once. However, this memory efficiency comes with the disadvantage that the array is accessed randomly, so cannot take advantage of a data cache smaller than the whole array.

As with all bucket sorts, performance depends critically on the balance of the buckets. In the ideal case of a balanced data set, each bucket will be approximately the same size. If the number m of buckets is linear in the input size n, each bucket has a constant size, so sorting a single bucket with an $O(n^2)$ algorithm like insertion sort has complexity $O(1^2) = O(1)$. The running time of the final insertion sorts is therefore $m \cdot O(1) = O(m) = O(n)$.

Choosing a value for m, the number of buckets, trades off time spent classifying elements (high m) and time spent in the final insertion sort step (low m). For example, if m is chosen proportional to $\sqrt{n}$, then the running time of the final insertion sorts is therefore $m \cdot O(\sqrt{n}^2) = O(n^{3/2})$.

In the worst-case scenarios where almost all the elements are in a few buckets, the complexity of the algorithm is limited by the performance of the final bucket-sorting method, so degrades to $O(n^2)$. Variations of the algorithm improve worst-case performance by using better-performing sorts such as quicksort or recursive flashsort on buckets which exceed a certain size limit.[2][3]

For $m = 0.1\ n$ with uniformly distributed random data, flashsort is faster than heapsort for all n and faster than quicksort for $n > 80$. It becomes about twice as fast as quicksort at $n = 10000$.[1] Note that these measurements were taken in the late 1990s, when memory hierarchies were much less dependent on cacheing.

Due to the *in situ* permutation that flashsort performs in its classification process, flashsort is not stable. If stability is required, it is possible to use a second array so elements can be classified sequentially. However, in this case, the algorithm will require $O(n)$ additional memory.

See also

- Interpolation search, using the distribution of items for searching rather than sorting

References

1. Neubert, Karl-Dietrich (February 1998). "The Flashsort1 Algorithm" (http://www.ddj.com/architect/184410496). *Dr. Dobb's Journal*. **23** (2): 123–125, 131. Retrieved 2007-11-06.

2. Neubert, Karl-Dietrich (1998). "The FlashSort Algorithm" (http://www.neubert.net/FSOIntro.html). Retrieved 2007-11-06.
3. Xiao, Li; Zhang, Xiaodong; Kubricht, Stefan A. (2000). "Improving Memory Performance of Sorting Algorithms: Cache-Effective Quicksort" (https://web.archive.org/web/20071102070431/http://jea.acm.org/ARTICLES/Vol5Nbr3/node4.html). *ACM Journal of Experimental Algorithmics*. **5**. CiteSeerX 10.1.1.43.736 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.43.736). doi:10.1145/351827.384245 (https://doi.org/10.1145%2F351827.384245). Archived from the original (http://jea.acm.org/ARTICLES/Vol5Nbr3/node4.html) on 2007-11-02. Retrieved 2007-11-06.

External links

- Implementations of Randomized Sorting on Large Parallel Machines (1992) (https://citeseer.ist.psu.edu/viewdoc/summary?doi=10.1.1.294.8609)
- Implementation of Parallel Algorithms (1992) (https://archive.org/details/DTIC_ADA253638)
- Visualization of Flashsort (http://home.westman.wave.ca/~rhenry/sort/#flashsort) Archived (https://web.archive.org/web/20110705025510/http://home.westman.wave.ca/~rhenry/sort/#flashsort) 2011-07-05 at the Wayback Machine